

Software Dependability

Java Microbenchmark Harness

What are we going to do today?

Write simple performance tests with JMH

<https://www.baeldung.com/java-microbenchmark-harness>

Useful Links

Java Microbenchmark Harness

<https://github.com/openjdk/jmh>

Performance testing

Performance Testing: the process of systematically evaluating performance features (e.g., execution time, throughput) of computing resources (e.g., CPU, memory) and software applications to find performance regressions

Benchmarking: performance testing activity executing an artificially generated workload against the system under test to compare different versions, configurations, or systems over time.

Microbenchmarking: benchmarking activity focused on small software components (e.g., Java methods, algorithms)

Microbenchmarking and JMH

A **microbenchmark** repeatedly execute a small code portion while gathering measurements of its execution time

Just-in-time compilation leads to microbenchmarks **performance variability** at the first phase of their execution (**warm-up phase**)

The **JVM** detects **frequently executed loops** or **methods**, **dynamically compiling** them into **optimized machine code**

For this reason, microbenchmarks are generally configured with a specific amount of **warm-up iterations**

Java Microbenchmark Harness: annotation-based framework that helps developers to build, run and analyze Java microbenchmarks

JMH Main Annotations

```
@Fork(1)
@Threads(1)
@Warmup(iterations = 5)
@Measurement(iterations = 5)
@OutputTimeUnit(TimeUnit.MICROSECONDS)

public class Basics {

    @Benchmark
    @BenchmarkMode(Mode.Throughput)
    @OutputTimeUnit(TimeUnit.SECONDS)
    public void measureThroughput() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
    }

    @Benchmark
    @BenchmarkMode(Mode.AverageTime)
    @OutputTimeUnit(TimeUnit.SECONDS)
    public void measureAvgTime() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
    }
}
```

Threads: specifies the number of threads that JMH will use to execute concurrently the benchmark

Fork: specifies the number of times that the benchmark process should be run

Warmup: specifies how many times the benchmark should run without measurement

Measurement: specifies the number of measurement iterations of the benchmark

All this annotation can be used at both method and class levels

Benchmark: specifies a benchmark method

The JMH Benchmark Modes

The benchmark modes tell JMH what to measure

Throughput is set as default

Throughput	Measures the number of operations per second, meaning the number of times per second your benchmark method could be executed.
Average Time	Measures the average time it takes for the benchmark method to execute (a single execution).
Sample Time	Measures how long time it takes for the benchmark method to execute, including max, min time etc.
Single Shot Time	Measures how long time a single benchmark method execution takes to run. This is good to test how it performs under a cold start (no JVM warm up).
All	Measures all of the above.

The JMH Benchmark Output Time Units

The benchmark output time units allow us to specify to JMH how we want to be benchmarks' results printed in.

For example, we could decide to measure the throughput and to show the results in seconds.

```
@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.SECONDS)
public void measureThroughput() throws Exception {
    TimeUnit.MILLISECONDS.sleep(100);
}
```

Time Units

NANOSECONDS
MICROSECONDS
MILLISECONDS
SECONDS
MINUTES
HOURS
DAYS

State Class and Scope

```
@Warmup(iterations = 5)
@Measurement(iterations = 5)
@OutputTimeUnit(TimeUnit.NANOSECONDS)

public class DefaultState {

    double x = Math.PI;

    @State(Scope.Thread)
    public static class MyState() {
        @Setup(Level.Trial)
        public void doSetup() {
            sum = 0;
        }
        @TearDown(Level.Trial)
        public void doTearDown() {
            System.out.println("do teardown");
        }
        public int a = 1;
        public int b = 2;
        public int sum;
    }

    @Benchmark @BenchmarkMode(Mode.Throughput)
    public void measure(MyState state) {
```

To initialize variables you don't want to be part of the benchmarked code but that you need, you can use **state** variables.

State: specifies the “State Class” that contains the state variables declarations.

The class must be **public**, and **static** if it's **nested**

To use the state class into the benchmark code, just use it as parameter

Scope:

Thread	Each thread has an instance of the State Object
Group	Each thread group has its own State Object
Benchmark	All threads share the same State Object

JMH Setup and TearDown

```
@State(Scope.Thread)
public static class MyState() {
    @Setup(Level.Trial)
    public void doSetup() {
        sum = 0;
    }
    @TearDown(Level.Trial)
    public void doTearDown() {
        System.out.println("do teardown");
    }
    public int a = 1;
    public int b = 2;
    public int sum;
}
```

Setup: tells JMH what method must be called to setup the state object before it is passed to the benchmark method.

TearDown: tells JMH what method must be called after the benchmark execution to clean the state object.

Of course, more than 1 method can be annotated with @Setup or @TearDown.

To tell JMH when specifically the methods should be called you can choose between the parameters:

Level.Trial	Call the method once for each full run of the benchmark method
Level.Iteration	Call the method once for each iteration of the benchmark method
Level.Invocation	Call the method once for each single call to the benchmark method

Multiple Parameters

```
@Warmup(iterations = 5)
@Measurement(iterations = 5)
@OutputTimeUnit(TimeUnit.NANOSECONDS)

public class Params {

    @Param({"1", "31", "65", "101", "103", })
    public int arg;

    @Param({"0", "1", "2", "4", "8", "16", "32"})
    public int certainly;

    @Benchmark
    public boolean bench() {
        return BigInteger.valueOf(arg).
            isProbablePrime(certainly);
    }
}
```

Param: specifies a list of values for a variable used by benchmarks.

The “bench” benchmark will be run with all the possible combinations of “arg” and “certainly”.

This configuration allows the developers to estimate the effect of a variable value on the benchmarked method efficiency.

Dead Code Elimination

```
@Benchmark
public void testMethod() {
    int a = 0;
    int b = 2;
    int sum = a + b;
}
```

=====

```
@Benchmark
public void testMethod() {
    int a = 0;
    int b = 2;
    int sum = a + b;
    return sum;
}
```

=====

```
@Benchmark
public void testMethod(Blackhole blackhole) {
    int a = 0;
    int b = 2;
    int sum = a + b;
    blackhole.consume(sum);
}
```

The JVM detects the “sum” is never used and considers it *dead code*. Also, it could consider “a + b” as dead code, consequently “a” and “b” useless.

No code is left, resulting in a misleading benchmark.

Making “testMethod” returning sum, the JVM cannot say if it is useless.

Otherwise, testMethod can take a Blackhole parameter and feed it with the sum variable

Constant Folding

```
@Benchmark
public void testMethod() {
    int a = 0;
    int b = 2;
    int sum = a + b;
    return sum;
}
```

```
=====
@Benchmark
public void testMethod(Blackhole blackhole) {
    return 2;
}
=====
```

```
@State(Scope.Thread)
public static class MyState() {
    public int a = 0;
    public int b = 2;
    public int sum;
}
```

```
@Benchmark
public void testMethod(MyState state) {
    int sum = state.a + state.b;
    return sum;
}
```

“sum” computation, since made on constants, will always lead to the same result. The JVM detect that and may reduce the whole benchmark to a single return statement (in the example).

The benchmark execution will be misleading!

To avoid the constant folding, a good practice is to use the State Class.

The constant can be declared in the State Class and retrieved by the benchmark methods, to make harder for the JVM to understand the values used by the benchmark come from constants.

What's Next?